Revelio is an email security app that uses CoreML to classify emails as phishing or benign, which also provides a sandbox to open files and links securely, and gives a dashboard view of threat intelligence based on the user's inbox. It reveals the threats hiding in your inbox. Existing email clients have spam filters, but they silently discard emails without any explanation. Revelio shows the user exactly what makes the email suspicious, so they build intuition over time rather than just depending on a black box. The primary users will be people with non-security backgrounds who deal with lots of emails every day such as healthcare workers, administrative staff, small business owners, and the secondary users will be people who want to be aware of the current phishing trends, such as security-aware individuals, students, educators, and researchers. So, the purpose of revelio is to be an intelligent security layer that analyzes incoming emails for threats, explains why something is risky in plain language, and lets users have control over how to deal with potentially insecure content.

Key features included in the app:

1. Email Classification: The app will be trained on Kaggle dataset via Create ML, and CoreML will be used to classify emails in the inbox. Filter options will be provided to view only phishing or benign emails. By default, all emails will be visible with phishing/benign clearly color-coded.
   Technical Notes: Use Create ML's text classifier on the Kaggle dataset to produce a .mlmodel file, integrate via CoreML's NLModel or MLModel API, seed mock inbox data from the dataset into SwiftData on first launch, and run classification asynchronously using Task and async/await to avoid blocking the UI.

2. Sandbox View: Let the users open the emails in a controlled environment where links are opened in WKWebView with a warning banner, attachment files show metadata only, and executables are blocked. A risk breakdown panel details the specific signals that triggered the classification.
   Technical Notes: Wrap WKWebView in a UIViewRepresentable for SwiftUI integration, inspect attachment MIME types and block executable extensions (.exe, .sh, .dmg) before rendering, store blocked action events in SwiftData, and display risk factors (suspicious keywords, sender domain, link mismatch) extracted during classification in a detail panel.

3. Compose Emails: Users can write emails which get stored in SwiftData (provides persistence for incoming/outgoing emails) and immediately scored by the model which can help expand the training dataset over time.

Technical Notes: Use SwiftData @Model for email persistence covering both incoming and outgoing messages, trigger the CoreML classifier on the composed text at send time, flag user-composed emails that score above a configurable risk threshold, and expose stored entries as potential retraining data in Settings.

4. Dashboard: The dashboard will present charts and stats on suspicious sender domains, top phishing keywords, blocked action logs, and risk trend over time.
   Technical Notes: Use Swift Charts for bar charts, pie charts, and line graphs, derive statistics from SwiftData queries over stored emails and blocked actions, and update charts reactively using SwiftData's @Query macro.

5. Settings/ About: Let the user decide the strictness level, optionally retrain/fine-tune the model on newer SwiftData entries.
   Technical Notes: Store user preferences with @AppStorage, expose a retraining flow using Create ML's programmatic API (MLTextClassifier) on device if feasible, or prompt the user to export data for offline retraining, display current model version and accuracy metrics in the About section.